
Experiment Title

Performance Analysis of Time-Shared vs. Space-Shared Virtual Machine Allocation Policies (Pure CLI Setup)

---

1. Objectives

- To compile and execute a CloudSim project purely from the **command line**.
- To understand the practical execution variance between **Time-Shared** and **Space-Shared** resource allocation policies.
- To parse and analyze metrics like **Cloudlet Finish Time** and **Context Execution Status**.

---

2. Lab Setup & Environment (10 Minutes)

Step 1: Download Required Files

1. Ensure Java JDK is installed (`java -version` and `javac -version` should work in your terminal).
2. Create a clean working folder on your desktop named `CloudSimLab`.
3. Download the official **CloudSim JAR** file (e.g., `cloudsim-3.0.3.jar` or any version you have) and drop it directly inside your `CloudSimLab` folder.

Step 2: Create the Source Code File

Inside the `CloudSimLab` folder, create a plain text file named **`App.java`** and paste the following complete, compilation-ready code:

```
//import org.cloudbus.cloudsim.*;
//import org.cloudbus.cloudsim.core.CloudSim;
//import java.text.DecimalFormat;
//import java.util.*;

import org.cloudbus.cloudsim.*;
import org.cloudbus.cloudsim.core.CloudSim;
import org.cloudbus.cloudsim.provisioners.PeProvisionerSimple;
import org.cloudbus.cloudsim.provisioners.RamProvisionerSimple;
import org.cloudbus.cloudsim.provisioners.BwProvisionerSimple;

import java.text.DecimalFormat;
import java.util.*;

public class App {
    public static void main(String[] args) {
        System.out.println("Starting Terminal-Based CloudSim Lab
Simulation...");
        try {
```

```
            // Step 1: Initialize CloudSim core engine
            int numUser = 1;
            Calendar calendar = Calendar.getInstance();
            CloudSim.init(numUser, calendar, false);

            // Step 2: Create Infrastructure Components
            Datacenter datacenter0 = createDatacenter("Datacenter_0");
            DatacenterBroker broker = new DatacenterBroker("Broker");
            int brokerId = broker.getId();

            // Step 3: Configure 2 Virtual Machines (VMs)
            List<Vm> vmlist = new ArrayList<Vm>();
            int vmid = 0;
            int mips = 1000;
            long size = 10000;
            int ram = 512;
            long bw = 1000;
            int pesNumber = 1; // Single-core VMs

            /*
             * EXERCISE CRITICAL STEP:
             * RUN 1: Set both to new CloudletSchedulerSpaceShared()
             * RUN 2: Set both to new CloudletSchedulerTimeShared()
             */
            Vm vm1 = new Vm(vmid++, brokerId, mips, pesNumber, ram, bw,
size, "Xen", new CloudletSchedulerSpaceShared());
            Vm vm2 = new Vm(vmid++, brokerId, mips, pesNumber, ram, bw,
size, "Xen", new CloudletSchedulerSpaceShared());

            vmlist.add(vm1);
            vmlist.add(vm2);
            broker.submitVmList(vmlist);

            // Step 4: Configure 4 Cloudlets (Tasks)
            List<Cloudlet> cloudletList = new ArrayList<Cloudlet>();
            int id = 0;
            long length = 40000; // 40,000 Millions of Instructions (MI)
            long fileSize = 300;
            long outputSize = 300;
            UtilizationModel utilizationModel = new UtilizationModelFull();

            for(int i = 0; i < 4; i++) {
                Cloudlet cloudlet = new Cloudlet(id++, length, pesNumber,
fileSize, outputSize, utilizationModel, utilizationModel,
utilizationModel);
                cloudlet.setUserId(brokerId);
                cloudlet.setVmId(i % 2); // Dispatches Cloudlets
dynamically alternating between VM 0 and VM 1
                cloudletList.add(cloudlet);
            }
            broker.submitCloudletList(cloudletList);

            // Step 5: Execute Simulation Run
            CloudSim.startSimulation();
            CloudSim.stopSimulation();

            // Step 6: Fetch Output
            List<Cloudlet> newList = broker.getCloudletReceivedList();
            printCloudletList(newList);

        } catch (Exception e) {
```

```
            e.printStackTrace();
        }
    }

    private static Datacenter createDatacenter(String name) throws
Exception {
        List<Host> hostList = new ArrayList<Host>();
        List<Pe> peList = new ArrayList<Pe>();
        peList.add(new Pe(0, new PeProvisionerSimple(2000))); // 2000 MIPS
CPU Core

        hostList.add(new Host(0, new RamProvisionerSimple(2048), new
BwProvisionerSimple(10000),
                1000000, peList, new VmSchedulerTimeShared(peList)));

        DatacenterCharacteristics characteristics = new
DatacenterCharacteristics(
                "x86", "Linux", "Xen", hostList, 10.0, 3.0, 0.05, 0.001,
0.0);

        return new Datacenter(name, characteristics, new
VmAllocationPolicySimple(hostList), new LinkedList<Storage>(), 0);
    }

    private static void printCloudletList(List<Cloudlet> list) {
        String indent = "    ";
        System.out.println("\n========== SIMULATION OUTPUT WINDOW
==========");
        System.out.println("Cloudlet ID" + indent + "STATUS" + indent + "VM
ID" + indent + "Time Taken" + indent + "Start Time" + indent + "Finish
Time");

        DecimalFormat dft = new DecimalFormat("###.##");
        for (Cloudlet cloudlet : list) {
            System.out.print(indent + cloudlet.getCloudletId() + indent +
indent);
            if (cloudlet.getCloudletStatus() == Cloudlet.SUCCESS) {
                System.out.println("SUCCESS" + indent + indent +
cloudlet.getVmId() + indent + indent +
dft.format(cloudlet.getActualCPUTime()) + indent + indent +
dft.format(cloudlet.getExecStartTime()) + indent + indent +
dft.format(cloudlet.getFinishTime()));
            }
        }
    }
}
```

Your `CloudSimLab` directory structural mapping should look exactly like this:

```
CloudSimLab/
  ├── cloudsim-3.0.3.jar
  └── App.java
```

---

3. Execution via Terminal (25 Minutes)

Run 1: Testing Space-Shared Policy

1. Open your terminal or command prompt.
2. Navigate directly into your `CloudSimLab` directory:

3. `cd Desktop/CloudSimLab`
4. Compile the application by linking the CloudSim jar file via the classpath (`-cp`):
    - **Windows (Command Prompt):**
    - `javac -cp "cloudsim-3.0.3.jar" App.java`
    - **Linux / macOS / PowerShell:**
    - `javac -cp "cloudsim-3.0.3.jar" App.java`

Take a Screenshot here - 1

5. Run the compiled application program:
    - **Windows (Command Prompt):**
    - `java -cp "cloudsim-3.0.3.jar;." App`
    - **Linux / macOS / PowerShell:**
    - `java -cp "cloudsim-3.0.3.jar:." App`

Take a Screenshot here - 2

6. Observe the output metrics table printed onto your screen.
7. **Sample Output:**

========== SIMULATION OUTPUT WINDOW ==========

Cloudlet ID	STATUS	VM ID	Time Taken	Start Time	Finish Time
0	SUCCESS	0	40	0.1	40.1
1	SUCCESS	1	40	0.1	40.1
2	SUCCESS	0	40	40.1	80.1
3	SUCCESS	1	40	40.1	80.1

Run 2: Testing Time-Shared Policy

1. Open `App.java` using any basic system text editor (Notepad, TextEdit, Nano, or VS Code).
2. Edit the Java program to change `new CloudletSchedulerSpaceShared()` to `new CloudletSchedulerTimeShared()`. Save and close the file.
3. Re-compile the application in your terminal window:

```
javac -cp "cloudsim-3.0.3.jar" App.java
```

Take a Screenshot here - 3

4. Execute the simulation run command again:
    - **Windows:** `java -cp "cloudsim-3.0.3.jar;." App`
    - **Linux/macOS:** `java -cp "cloudsim-3.0.3.jar:." App`

Take a Screenshot here - 4

Sample output:

========== SIMULATION OUTPUT WINDOW ==========

Cloudlet ID	STATUS	VM ID	Time Taken	Start Time	Finish Time
0	SUCCESS	0	80	0.1	80.1
2	SUCCESS	0	80	0.1	80.1
1	SUCCESS	1	80	0.1	80.1
3	SUCCESS	1	80	0.1	80.1

---

4. Data Logs & Observations

Observation 1: Space-Shared Policy

*(**Expected behavior**: Cloudlets 0 and 1 will execute immediately. Cloudlets 2 and 3 will have to wait for the first batch to finish, resulting in a delayed Start Time).*

Observation 2: Time-Shared Policy

*(**Expected behavior**: All 4 cloudlets will start concurrently at `Start Time = 0.0`. However, because the single CPU core is shared, the total execution time for each cloudlet will double).*
